

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroller

Adzkia Anfariza Rizqika - 5024241003

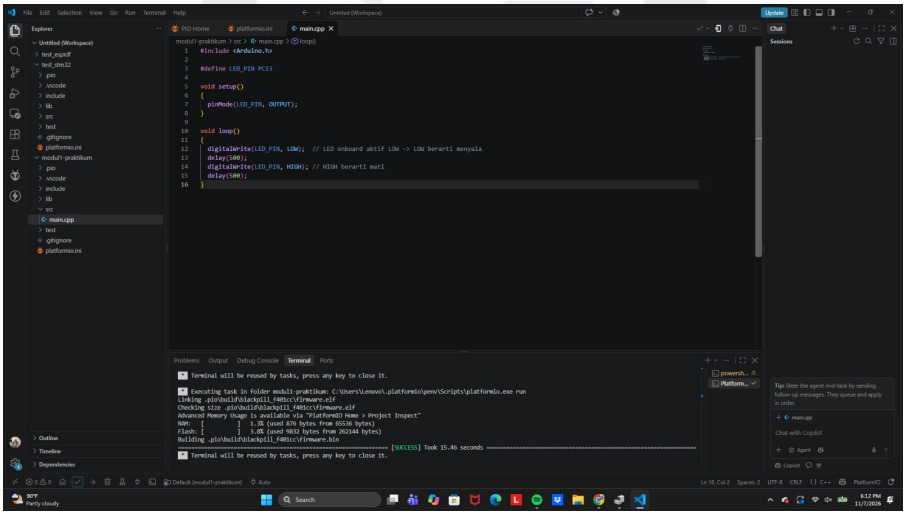
2026

1 Langkah-Langkah Percobaan

Bagian ini menjelaskan tahapan yang dilakukan selama praktikum Modul 1 terkait dasar Input/Output pada mikrokontroler. Percobaan menggunakan board STM32 Blackpill dan ESP32 dengan *environment* PlatformIO di Visual Studio Code.

1.1 Percobaan 1: Output Digital (Blink LED) pada STM32

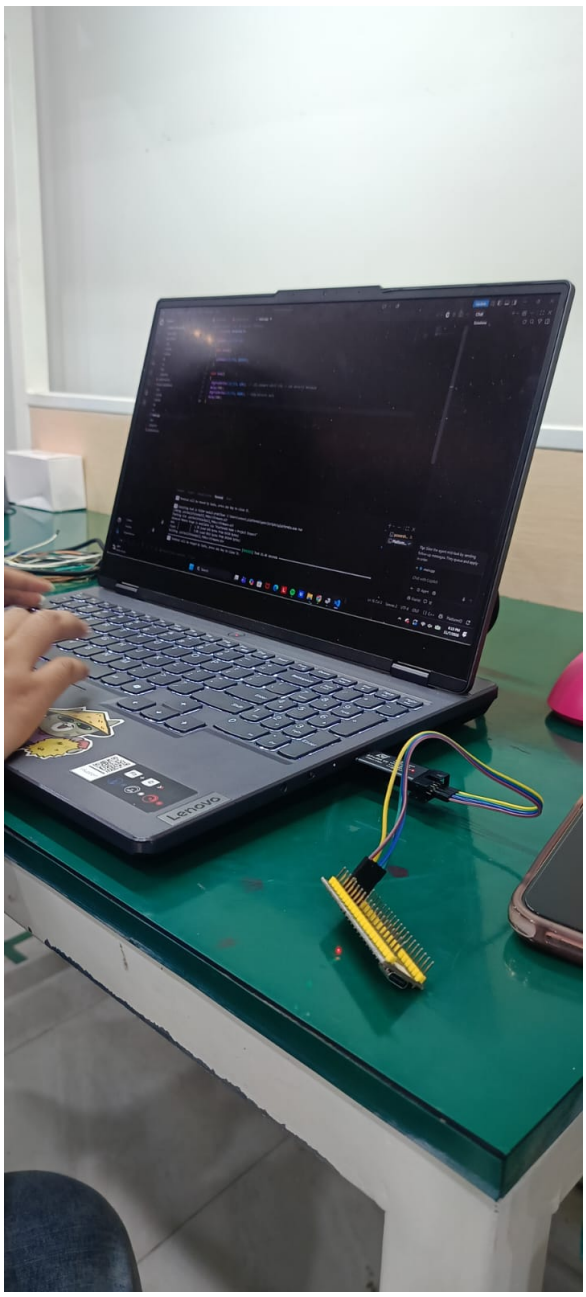
1. Membuka Visual Studio Code dan membuat *project* baru menggunakan ekstensi PlatformIO. Board yang dipilih adalah `blackpill_f401cc` dengan *framework* Arduino.
2. Mengonfigurasi file `platformio.ini` dengan menambahkan pengaturan protokol *upload* menggunakan `stlink` (`upload_protocol = stlink`).
3. Menuliskan program pada `main.cpp` untuk mengonfigurasi pin `PC13` sebagai `OUTPUT` dan memberikan logika `LOW` dan `HIGH` secara bergantian menggunakan fungsi `delay(500)`.
4. Menghubungkan STM32 Blackpill ke komputer menggunakan kabel ST-Link[cite: 9, 11].
5. Melakukan proses *Build* dan *Upload* program ke mikrokontroler.
6. Mengamati hasil keluaran berupa LED *onboard* (PC13) yang berkedip sesuai dengan delay yang telah diprogram[cite: 2, 8].



Gambar 1: Kode program Blink LED STM32 dan proses upload yang berhasil.



Gambar 2: STM32 menyala setelah di-upload.

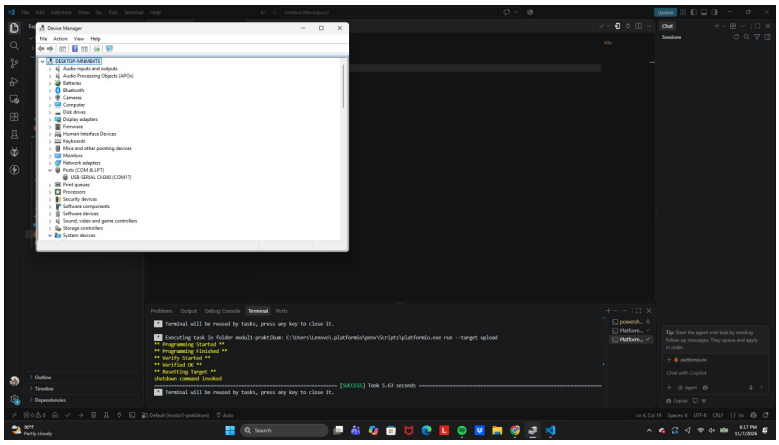


Gambar 3: Koneksi fisik STM32 via ST-Link.

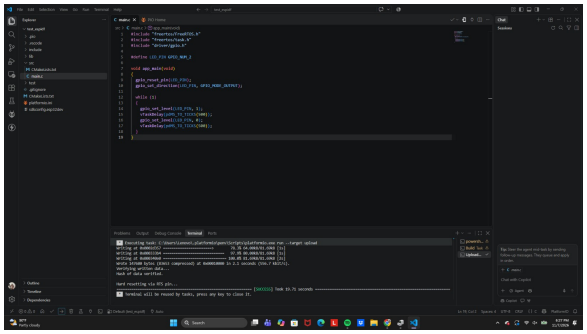
1.2 Percobaan 2: Output Digital pada ESP32 menggunakan ESP-IDF

1. Membuat *project* baru di PlatformIO dengan board `esp32dev` dan mengatur *framework* menggunakan `espidf` pada file `platformio.ini`.
2. Memeriksa koneksi ESP32 di *Device Manager* untuk memastikan port USB terbaca (misalnya `COM17`)[cite: 5].
3. Menuliskan program berbahasa C pada `main.c`. Pin GPIO 2 diinisialisasi sebagai output menggunakan `gpio_set_direction`.
4. Menggunakan fungsi FreeRTOS seperti `vTaskDelay` untuk memberikan jeda waktu perubahan status LED.

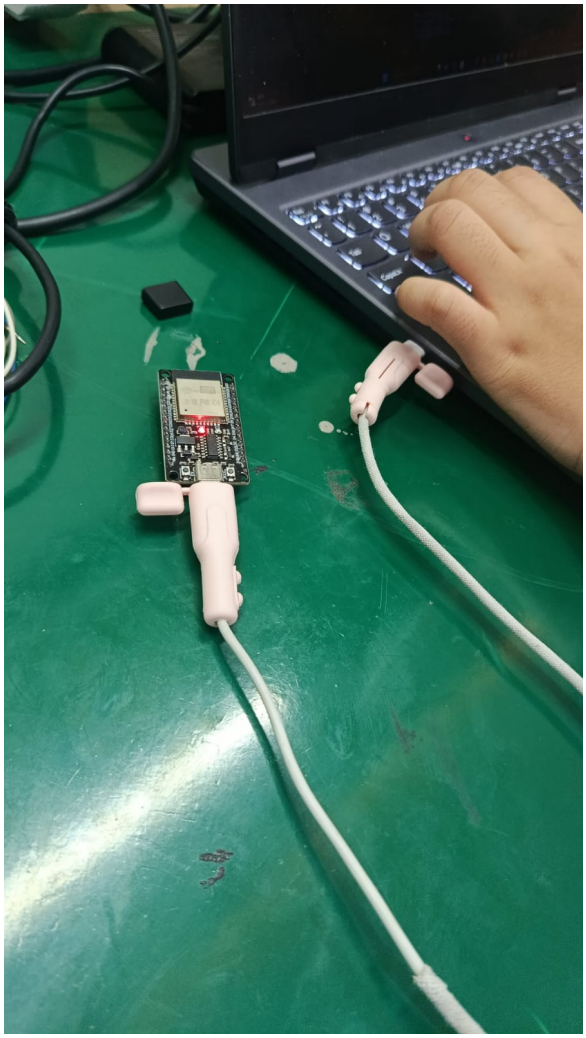
5. Melakukan *upload* program dan mengamati LED *onboard* pada ESP32 berkedip[cite: 3, 4, 6].



Gambar 4: Pengecekan port komunikasi (COM) pada Device Manager.



Gambar 5: Kode program ESP-IDF untuk GPIO 2.



Gambar 6: ESP32 terhubung dan beroperasi.

1.3 Percobaan 3: Input Digital (Pull-up dan Pull-down Eksternal & Internal)

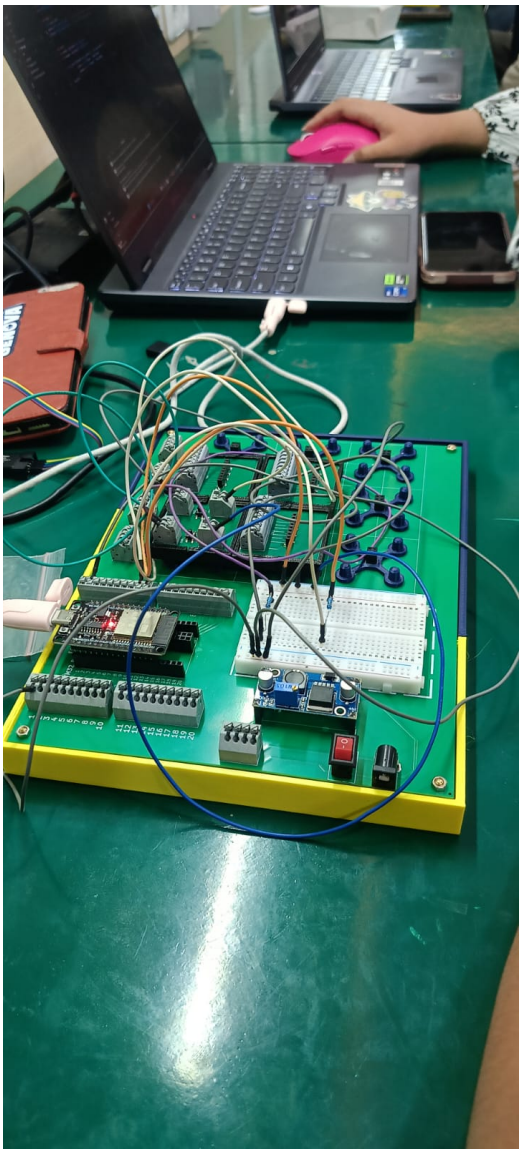
1. Mengubah *framework* project ESP32 kembali menjadi Arduino[cite: 20].

2. Merangkai sirkuit pada *breadboard*, menghubungkan *push button* ke pin GPIO 32 (untuk pull-up) dan GPIO 33 (untuk pull-down). Percobaan pertama menggunakan resistor yang dipasang secara perangkat keras (eksternal)[cite: 18].

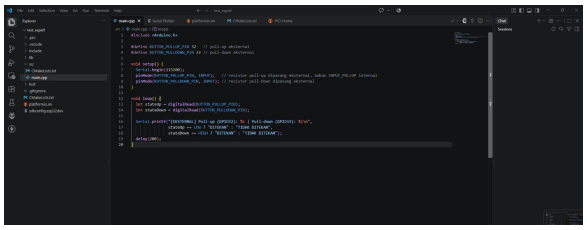
3. Menulis program untuk membaca status pin menggunakan `digitalRead()` dan menampilkannya di Serial Monitor[cite: 12, 16, 17].

4. Setelah pengujian resistor eksternal berhasil, memodifikasi program untuk menggunakan konfigurasi resistor internal mikrokontroler dengan mengubah inisialisasi pin menjadi `INPUT_PULLUP` dan `INPUT_PULLDOWN`[cite: 13, 15].

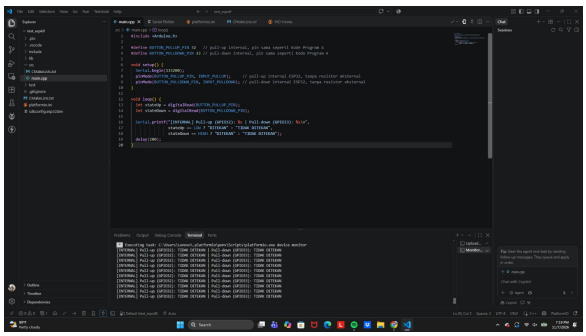
5. Menekan tombol secara fisik dan mengamati perubahan status "DITEKAN" dan "TIDAK DITEKAN" pada Serial Monitor.



Gambar 7: Rangkaian fisik uji coba input digital pada breadboard.



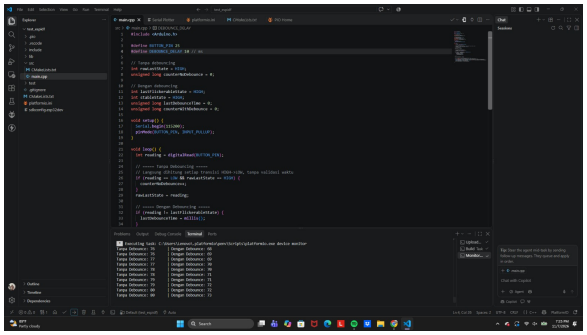
Gambar 8: Kode pengujian pull-up/pull-down eksternal.



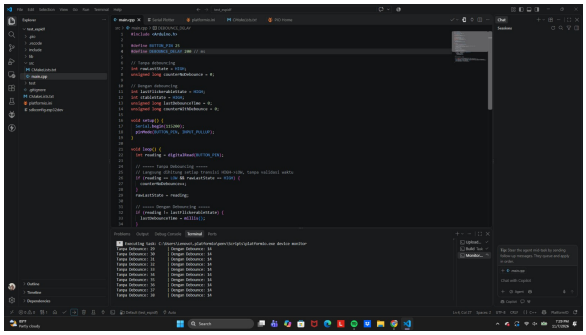
Gambar 9: Kode dan output pull-up/pull-down internal.

1.4 Percobaan 4: Debouncing Tombol

1. Membuat program penghitung (*counter*) penekanan tombol menggunakan pin GPIO 25 dengan konfigurasi `INPUT_PULLUP`.
2. Menyusun kode untuk membandingkan dua kondisi: pembacaan transisi tombol secara langsung (tanpa *debouncing*) dan pembacaan menggunakan *software debouncing* memanfaatkan fungsi `millis()`.
3. Melakukan variasi pengujian batas waktu (*debounce delay*), mulai dari 10 ms[cite: 26, 27, 28, 29], 50 ms[cite: 31], hingga 200 ms[cite: 24, 25].
4. Menekan tombol berulang kali[cite: 30] dan membandingkan jumlah hitungan variabel `counterNoDebounce` dan `counterWithDebounce` pada Serial Monitor.



Gambar 10: Uji coba debouncing dengan delay 10 ms.



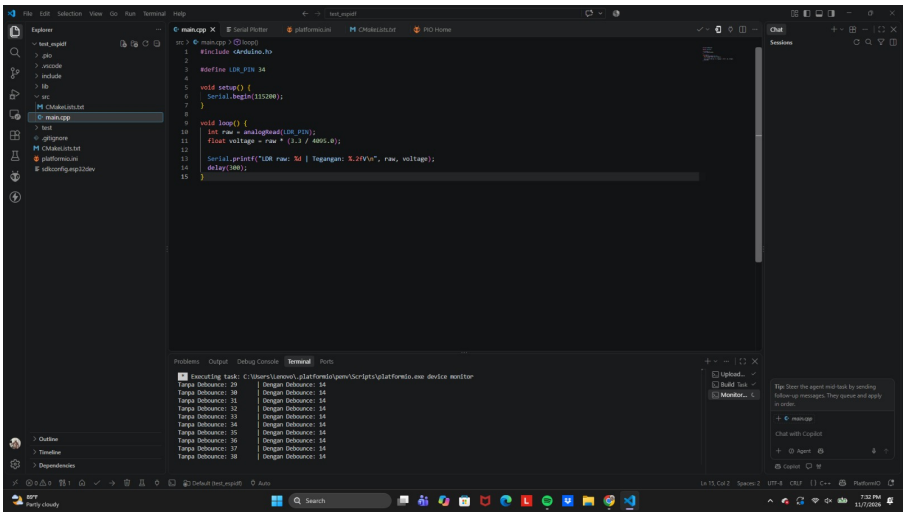
Gambar 11: Uji coba debouncing dengan delay 200 ms.

1.5 Percobaan 5: Input Analog menggunakan Sensor LDR

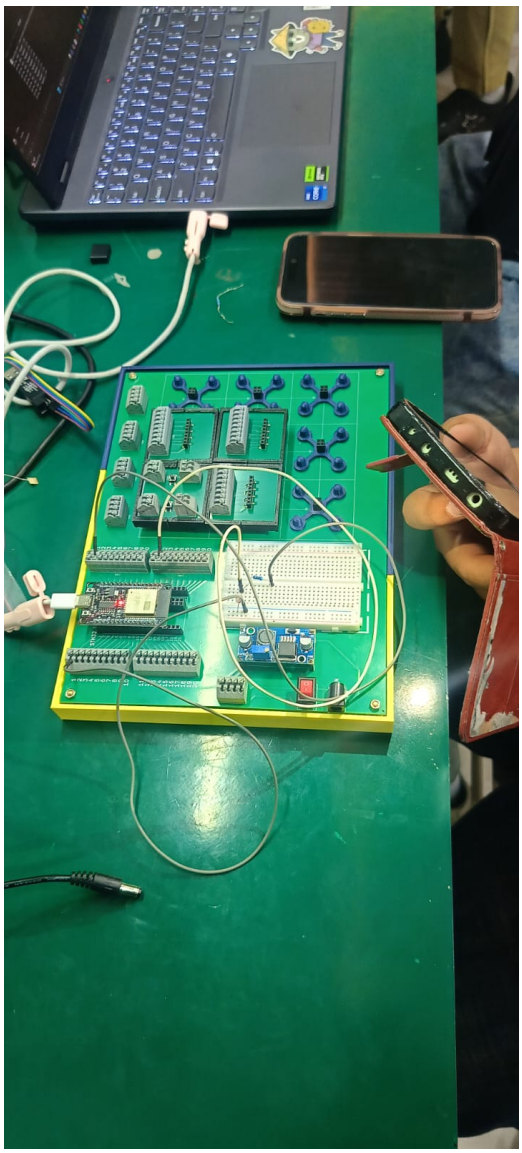
1. Merangkai sensor cahaya (LDR) pada *breadboard* sebagai pembagi tegangan (voltage divider) dan menghubungkan pin output LDR ke pin ADC ESP32 (GPIO 34).
2. Menulis program untuk membaca nilai tegangan analog (*raw* ADC 12-bit) menggunakan fungsi `analogRead()`[cite: 22, 23, 36].
3. Mengonversi nilai resolusi (0-4095) ke dalam perkiraan nilai tegangan sebenarnya (0-3.3 Volt).
4. Menampilkan nilai *raw* dan tegangan ke Serial Monitor.

5. Melakukan pengujian dengan memberikan perlakuan khusus: menutupi sensor LDR dengan tangan (kondisi gelap)[cite: 32, 33, 35, 37] dan memberikan cahaya (kondisi terang)[cite: 34].

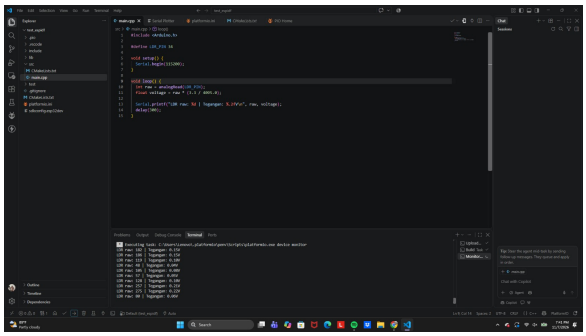
6. Mengamati perubahan nilai pada Serial Monitor berdasarkan intensitas cahaya yang diterima LDR.



Gambar 12: Kode pembacaan nilai LDR menggunakan ADC ESP32.



Gambar 13: Pengujian dengan membuka sensor LDR (terang).



Gambar 14: Output Serial Monitor saat sensor ditutup.

2 Analisis Hasil Percobaan

Bagian ini memuat evaluasi dan pembahasan dari hasil praktikum yang telah dilakukan, mulai dari pengontrolan I/O digital dasar hingga pembacaan sensor analog.

2.1 Analisis Percobaan 1: Output Digital (Blink LED) pada STM32

Pada percobaan pertama, LED *onboard* STM32 (terhubung pada pin PC13) berhasil dikendalikan menggunakan *framework* Arduino. Berdasarkan kode yang dieksekusi, LED *onboard* STM32 memiliki konfigurasi *active-low*, yang berarti pemberian logika `LOW` menggunakan `digitalWrite()` akan membuat LED menyala, dan logika `HIGH` akan mematikannya[cite: 2]. Fungsi `delay(500)` bekerja sesuai dengan teori untuk memberikan jeda penahanan status selama 500 milidetik sebelum mengeksekusi baris kode berikutnya. Penggunaan PlatformIO terbukti mempermudah pengaturan protokol ST-Link tanpa perlu menggunakan *software flasher* eksternal[cite: 8].

2.2 Analisis Percobaan 2: Output Digital pada ESP32 menggunakan ESP-IDF

Pengendalian output pada ESP32 memberikan perspektif berbeda karena menggunakan *framework* bawaan Espressif (ESP-IDF) yang berbasis sistem operasi waktu nyata (FreeRTOS)[cite: 3]. Alih-alih menggunakan `pinMode()` dan `delay()` seperti pada Arduino, inisialisasi dilakukan menggunakan `gpio_set_direction()` dan jeda diatur dengan `vTaskDelay()`[cite: 4]. Penggunaan `pdMS_TO_TICKS(500)` memvalidasi prinsip RTOS di mana mikrokontroler menghitung *tick* dari *system clock* untuk menunda suatu tugas (*task*), sehingga memungkinkan eksekusi proses lain di latar belakang tanpa memblokir seluruh sistem (berbeda dengan *blocking delay* standar)[cite: 3, 4].

2.3 Analisis Percobaan 3: Input Digital (Pull-up dan Pull-down)

Analisis dari percobaan ini membuktikan pentingnya resistor *pull-up* atau *pull-down* untuk mencegah kondisi *floating* (status logika tidak menentu) pada mikrokontroler saat sakelar terbuka[cite: 13]. Pengujian menggunakan perangkat keras (resistor eksternal) dan fitur internal mikrokontroler menunjukkan hasil pembacaan yang sama-sama stabil[cite: 14, 16]. Namun, penggunaan parameter `INPUT_PULLUP` dan `INPUT_PULLDOWN` pada fungsi `pinMode()` terbukti jauh lebih efisien karena meminimalisasi penggunaan komponen eksternal dan penyambungan kabel fisik pada *breadboard*. Kelebihan ESP32 yang mendukung *pull-down* internal juga terlihat bekerja dengan baik, fitur yang umumnya tidak tersedia secara bawaan pada mikrokontroler AVR standar[cite: 13, 15].

2.4 Analisis Percobaan 4: Debouncing Tombol

Karakteristik mekanis dari *push button* terbukti menyebabkan fenomena *bouncing*, yakni transisi tegangan yang berosilasi cepat sebelum mencapai kondisi stabil. Hal ini dibuktikan oleh nilai `counterNoDebounce` yang melonjak tidak beraturan meskipun tombol hanya ditekan satu kali[cite: 24, 26]. Implementasi *software debouncing* menggunakan pembacaan `millis()` efektif mengatasi masalah ini karena program hanya akan mengeksekusi perhitungan (`counterWithDebounce`) apabila status tombol tidak berubah melewati batas waktu transisi (*debounce delay*)[cite: 28, 29]. Dari pengujian berbagai rentang *delay* (10ms, 50ms, dan 200ms), disimpulkan bahwa *delay* terlalu kecil (seperti 10ms) mungkin masih rentan terhadap *bounce* yang ekstrem, sedangkan *delay* terlalu lama (seperti 200ms) meredam osilasi dengan sempurna namun dapat membuat tombol terasa kurang responsif jika ditekan secara berurutan dalam waktu yang sangat cepat[cite: 27, 31].

2.5 Analisis Percobaan 5: Input Analog menggunakan Sensor LDR

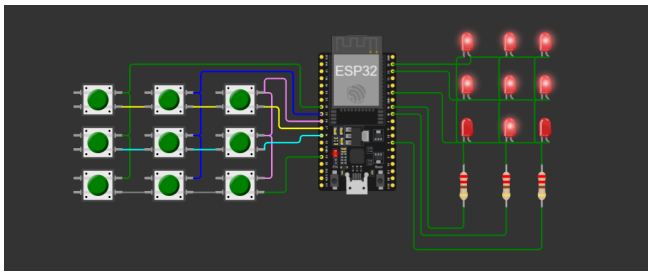
Percobaan ini mengevaluasi kinerja konverter analog-ke-digital (ADC) pada ESP32 yang memiliki resolusi tinggi sebesar 12-bit. Secara matematis, resolusi ini membagi tegangan referensi (3,3V) ke dalam 4096 tingkatan nilai (0 hingga 4095)[cite: 22]. Konversi dari nilai *raw* ke satuan volt di dalam kode program (`voltage = raw * (3.3 / 4095.0)`) terbukti berhasil merepresentasikan tegangan sesungguhnya yang masuk ke pin GPIO 34[cite: 23, 36]. Dari observasi fisik, konfigurasi LDR dan resistor seri bertindak sebagai rangkaian pembagi tegangan (*voltage divider*)[cite: 33]. Saat sensor ditutup (keadaan gelap), resistansi LDR meningkat drastis yang menyebabkan jatuh tegangan (drop) pada pin pembacaan mendekati nol (nilai raw anjlok hingga $\sim$48-182, merepresentasikan tegangan sekitar 0,04V - 0,15V)[cite: 35]. Sebaliknya, saat diterangi cahaya, resistansi LDR menurun dan tegangan yang terbaca meningkat hingga melampaui nilai *raw* 3300 (sekitar 2,7V)[cite: 34]. Ini mengonfirmasi

bahwa nilai tegangan pembacaan berbanding lurus dengan intensitas cahaya berdasarkan konfigurasi pembagi tegangannya.

3 Hasil Tugas Modul

3.1 Tugas Simulasi

Tugas modul pertama, menyalakan 9 LED berdasarkan posisi baris dan kolom buttonnya.



Gambar 15: Simulasi tugas modul pada Wokwi

Berikut adalah kode yang digunakan,

```
#include <Arduino.h>

const int rowPinsButton[3] = {13, 14, 27};
const int colPinsButton[3] = {26, 25, 33};

const int rowPinsLed[3] = {21, 22, 23};
const int colPinsLed[3] = {4, 18, 19};

const unsigned long DEBOUNCE_DELAY = 50;

bool ledState[3][3] = {false};
int lastButtonState[3][3];
int stableButtonState[3][3];
unsigned long lastDebounceTime[3][3] = {0};

void setup() {
  Serial.begin(115200);

  for (int i = 0; i < 3; i++) {
    pinMode(rowPinsButton[i], OUTPUT);
    digitalWrite(rowPinsButton[i], HIGH);

    pinMode(colPinsButton[i], INPUT_PULLUP);
  }

  for (int i = 0; i < 3; i++) {
```

```
    pinMode(rowPinsLed[i], OUTPUT);
    digitalWrite(rowPinsLed[i], LOW);

    pinMode(colPinsLed[i], OUTPUT);
    digitalWrite(colPinsLed[i], HIGH);
  }

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      lastButtonState[r][c] = HIGH;
      stableButtonState[r][c] = HIGH;
    }
  }
}

void loop() {
  unsigned long currentTime = millis();

  for (int r = 0; r < 3; r++) {
    digitalWrite(rowPinsButton[r], LOW);

    for (int c = 0; c < 3; c++) {
      int reading = digitalRead(colPinsButton[c]);

      if (reading != lastButtonState[r][c]) {
        lastDebounceTime[r][c] = currentTime;
      }

      if ((currentTime - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
        if (reading != stableButtonState[r][c]) {
          stableButtonState[r][c] = reading;

          if (stableButtonState[r][c] == LOW) {
            ledState[r][c] = !ledState[r][c];

            Serial.printf("Tombol Ditekan -> Baris: %d, Kolom: %d | Status LED: %s\n",
                          r + 1, c + 1, ledState[r][c] ? "MENYALA" : "MATI");
          }
        }
      }

      lastButtonState[r][c] = reading;
    }
```

```
    digitalWrite(rowPinsButton[r], HIGH);
  }

  for (int r = 0; r < 3; r++) {
    digitalWrite(rowPinsLed[r], HIGH);

    for (int c = 0; c < 3; c++) {
      if (ledState[r][c]) {
        digitalWrite(colPinsLed[c], LOW);
      } else {
        digitalWrite(colPinsLed[c], HIGH);
      }
    }

    delay(2);

    digitalWrite(rowPinsLed[r], LOW);
    for (int c = 0; c < 3; c++) {
      digitalWrite(colPinsLed[c], HIGH);
    }
  }
}
```

3.2 Pertanyaan Analisis

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing

   Meskipun pada satu waktu hanya satu tombol yang ditekan, *debouncing* tetap mutlak diperlukan karena sifat mekanis dari sakelar (*push button*). Di dalam sakelar terdapat bilah logam elastis yang saling bersentuhan saat ditekan. Sifat mekanis ini menyebabkan terjadinya osilasi atau pantulan tegangan yang sangat cepat (naik-turun antara HIGH dan LOW) selama beberapa milidetik sebelum akhirnya benar-benar stabil pada satu kondisi. Tanpa adanya fungsi *debouncing* (seperti validasi waktu menggunakan *millis()*), mikrokontroler akan mendeteksi osilasi transisi tersebut sebagai pulsa penekanan tombol berkali-kali secara keliru (seperti yang terlihat pada Percobaan 4), sehingga sistem mencatat jumlah penekanan atau mengubah status LED secara tidak beraturan.

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

   Pada rancangan sistem ini, pembacaan kolom tombol menggunakan konfigurasi `INPUT_PULLUP` internal mikrokontroler.

   (a) **Kondisi saat tombol tidak ditekan:** Pin kolom berada pada kondisi `HIGH` karena resistor

pull-up menghubungkan jalur pin ke sumber tegangan internal (V_{CC}).

(b) **Mengapa tidak boleh *floating*:** Jika pin dibiarkan dalam kondisi *floating* (tanpa terhubung ke logika tetap saat tombol terbuka), pin akan bertindak sebagai antena yang sangat sensitif terhadap derau elektromagnetik (*electrical noise*) di sekitarnya. Hal ini menyebabkan pembacaan status pin menjadi acak dan tidak menentu (*random trigger*), sehingga mikrokontroler dapat mendeteksi seolah-olah tombol sedang ditekan padahal tidak ada aktivitas fisik sama sekali.

3. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

   (a) **Digital Input (Tombol):** Hanya mengenali dua kondisi diskrit yang tegas, yaitu logika `HIGH` (mendekati tegangan penuh, misal 3,3V) atau `LOW` (mendekati 0V). Nilai ini dibaca menggunakan fungsi `digitalRead()`.

   (b) **Analog Input (ADC - Percobaan 5):** Menggunakan konverter *Analog-to-Digital* (ADC) dengan resolusi tertentu (misalnya 12-bit pada ESP32 yang memiliki 4096 tingkatan nilai dari 0 hingga 4095). Fungsi `analogRead()` mengukur tingkat tegangan kontinu yang masuk pada pin secara proporsional.

   (c) **Mengapa ADC dapat merepresentasikan lebih dari dua kondisi:** Karena ADC memetakan rentang tegangan kontinu (dari 0V hingga 3,3V) ke dalam cakupan nilai numerik yang luas (0 sampai 4095). Perubahan resistansi pada sensor seperti LDR akibat variasi intensitas cahaya akan menghasilkan gradasi tegangan yang halus dan bervariasi, memungkinkan sistem membaca banyak tingkatan kondisi cahaya (mulai dari sangat gelap, redup, hingga sangat terang), bukan sekadar batas biner aktif/tidak aktif.

4 Kesimpulan

Berdasarkan serangkaian percobaan yang telah dilakukan pada Modul 1 mengenai dasar Input/Output pada mikrokontroler (STM32 Blackpill dan ESP32), dapat disimpulkan bahwa:

1. Pengendalian output digital (Blink LED) berhasil dilakukan menggunakan platform *development* PlatformIO, baik menggunakan kerangka kerja Arduino maupun ESP-IDF (FreeRTOS) dengan menerapkan pengaturan logika *active-low* serta pengelolaan jeda waktu yang sesuai.

2. Konfigurasi input digital memerlukan resistor *pull-up* atau *pull-down* (baik eksternal maupun internal seperti `INPUT_PULLUP`) untuk mencegah kondisi *floating* yang dapat menimbulkan derau atau pembacaan status tombol yang tidak stabil.

3. Fenomena *bouncing* pada sakelar mekanis dapat diatasi secara efektif melalui implementasi *software debouncing* berbasis fungsi waktu *millis()*, memastikan pembacaan status tombol benar-benar stabil sebelum dieksekusi oleh sistem.

4. Pembacaan sensor analog menggunakan ADC (Konverter Analog-ke-Digital) pada ESP32 dengan resolusi 12-bit mampu mengonversi variasi tegangan kontinu dari sensor pembagi tegang-

an (seperti LDR) menjadi rentang nilai numerik yang proporsional terhadap intensitas cahaya, jauh melampaui keterbatasan batas diskrit biner digital (HIGH/LOW).